

LLM-Enhanced Multimodal Product Recommendation with Review Aspect-Specific Knowledge Graphs

RA-GARK 專案交接文件

2026 年 8 月 17 日

1 Environment

- 建議環境：py311_cuda126。
- 所有 Python 指令皆從 Code/ 目錄執行。
- 目前 repo 沒有固定提供完整 environment file；若在新機器接手，請先安裝 Python 依賴，再執行 smoke test 確認環境與資料路徑。

程式碼架構

Code/

```
config.py
data.py
kg_loader.py
model.py
losses.py
evaluate.py
utils.py
train_ragark.py
run_ablations.py
run_main_benchmark.py
case_study.py
smoke_test.py
baselines/
data/
handoff/
```

```

relation_init/
results/
runs/

```

檔案或資料夾	說明
config.py	實驗設定檔，包含 data path、模型參數、訓練參數與 ablation flags。
data.py	載入 review interaction、建立 KG index、建立 LightGCN adjacency。
kg_loader.py	canonical KG loader；只有 canonical KG branch 或檢查工具會用到。
model.py	RA-GARK 模型架構。
losses.py	BPR loss、aspect-level contrastive loss、user cross-view contrastive loss。
evaluate.py	Top-K recommendation evaluation。
train_ragark.py	RA-GARK 主模型訓練與測試。
run_ablations.py	消融實驗與 sensitivity 實驗。
run_main_benchmark.py	baseline 與 RA-GARK 主比較實驗。
case_study.py	rationale attention case study。
smoke_test.py	快速檢查 data、KG、model forward/backward、evaluation 是否能跑通。
baselines/	KGAT、KGCL、KGRec、MCCLK baseline。
relation_init/	canonical KG initialization 的 archived experiment。
results/	最新輸出結果，例如 main benchmark、ablation、case study。
runs/	checkpoint、best ledger、archive CSV。

Smoke Test

使用方式：

```

cd Code
python smoke_test.py

```

`smoke_test.py` 的目的只是確認環境與主要 code path 可正常執行。它不是正式實驗，也不是論文數字來源。

2 Datasets

- 主要資料來源：Amazon Reviews 2023。
- 使用類別：Books。
- 原始 review 來源：<https://amazon-reviews-2023.github.io/>
- 目前主實驗使用的是已 preprocess 完成的檔案，位於 `Code/data/`。

目前實際使用的 data

檔案	用途
<code>data/reviews_30_20.pkl</code>	主要 interaction data。所有 RA-GARK、baseline、case study 預設都會讀這個檔案。
<code>data/df_edges_item_aspect1.csv</code>	預設 item-aspect KG，對應 <code>Config.kg_path</code> 。
<code>data/df_edges_all_aspect1.csv</code>	raw KG input，供 <code>kg_clean.py</code> 產生 canonical KG。
<code>data/kg_canonical.csv</code>	canonical KG；只有 <code>use_canonical_kg=True</code> 或 <code>relation_init/</code> 會使用。
<code>data/kg_relation_map.json</code>	raw relation 到 canonical relation 的 mapping，用於 audit。
<code>data/kg_canonical_report.txt</code>	canonical KG 清理報告。

主要 config 參數

參數名稱	說明	值範例
<code>interaction_path</code>	preprocess 後的 review interaction 檔案。	<code>data/reviews_30_20.pkl</code>
<code>kg_path</code>	預設 item-aspect KG。	<code>data/df_edges_item_aspect1.csv</code>
<code>canonical_kg_path</code>	canonical KG 檔案。	<code>data/kg_canonical.csv</code>

<code>use_canonical_kg</code>	是否改用 canonical KG。預設為 False False。
<code>kg_top_freq_pct</code>	legacy KG 中要剪掉的高頻 aspect 比 例。 0.02
<code>num_aspects</code>	RA-GARK aspect slots 數量。 4
<code>embedding_dim</code>	embedding 維度。 128
<code>eval_k</code>	evaluation 的主要 Top-K。 20

3 Preprocess

目前 repo 內保留的是已 preprocess 完成、可直接訓練的資料。正式接手時請先確認 Code/data/ 內上述資料皆已準備完成，再依照以下順序執行。

Preprocess 做的事情

目前主訓練資料 `reviews_30_20.pkl` 已完成以下處理：

- 過濾 review interaction，只保留可用欄位。
- 依照 like 欄位保留 positive interactions 作為推薦訓練資料。
- 對 user 與 item 重新 label encoding，產生 `user_idx` 與 `item_idx`。
- 使用 user-stratified split 產生 train、validation、test。
- 訓練時使用 `data/df_edges_item_aspect1.csv` 建立 item-aspect KG index。
- KG 會剪掉高頻 noisy aspects 與 `Config.kg_stopwords` 中的 stopwords。

如果要重新整理 canonical KG，可執行：

```
python kg_clean.py
```

此步驟會以 `data/df_edges_all_aspect1.csv` 為 input，輸出：

- `data/kg_canonical.csv`
- `data/kg_relation_map.json`
- `data/kg_canonical_report.txt`

1. 執行 RA-GARK

執行主模型：

```
python train_ragark.py
```

此指令會使用 config.py 中的預設設定進行訓練與 test evaluation。預設流程包含：

- 載入 reviews_30_20.pkl
- 建立 item-aspect KG index
- 建立 LightGCN adjacency
- 使用 RA-GARK 訓練推薦模型
- 以 validation NDCG 選擇 best checkpoint
- 載入 best checkpoint 並輸出 test metrics

2. 執行消融實驗

快速檢查版：

```
python run_ablations.py --mode minimal --reuse
```

論文表格版本：

```
python run_ablations.py --mode paper --reuse
```

sensitivity 實驗：

```
python run_ablations.py --mode sensitivity --reuse
```

主要輸出：

- results/ablation_results_<mode>.csv
- runs/archive/ablation_results_<mode>_<timestamp>.csv
- runs/best/
- runs/best_ledger.json

3. 執行 Baseline 模型

各 baseline 由 `run_main_benchmark.py` 統一執行：

```
python run_main_benchmark.py
```

包含模型：

- MCCLK
- KGCL
- KGAT
- KGRec
- LightGCN
- RA-GARK

主要輸出：

- `results/main_benchmark_results.csv`
- `runs/archive/main_benchmark_results_<timestamp>.csv`

4. Case Study

執行：

```
python case_study.py
```

用途：輸出 RA-GARK softmax rationale attention 的案例，觀察同一商品在不同 user 下的 aspect attention 是否不同。

主要輸出：

- `results/case_study.txt`
- `results/case_study.csv`

5. 結果檔案對應

檔案	說明
results/main_benchmark_results.csv	主模型與 baseline 的比較結果。
results/ablation_results_paper.csv	論文消融實驗表格主要來源。
results/ablation_results_full.csv	較完整的消融結果。
results/case_study.txt	human-readable case study。
results/case_study.csv	machine-readable case study attention weights。
runs/best_ledger.json	各 config 的 best-ever metrics ledger。

注意事項

- 不要任意覆蓋 runs/best/ 或 runs/best_ledger.json，除非明確要重新更新 best-run ledger。
- relation_init/ 是 archived experiment，除非要重訪 canonical KG initialization branch，否則不建議修改。
- 若只想確認環境，請執行 smoke_test.py；若要取得正式實驗數字，請執行 run_ablations.py 或 run_main_benchmark.py。
- 編譯本交接文件：

```
cd Code/handoff
```

```
latexmk -xelatex -interaction=nonstopmode main.tex
```